HDOC Day-63

Missing Number in an Array Using Pointers

Problem Statement

Read the size and elements of a variable-length integer array containing distinct values from 0 to size with exactly one number missing. Pass the array by address to missing_number(), and display the missing value.

Algorithm

1. Start.

2. Read size and validate that it is positive.

3. Declare a variable-length integer array nums[size].

4. Read size distinct integers; validate each value lies between 0 and size.

5. Call missing_number(nums, size) using the array address.

6. Compute expected = size * (size + 1) / 2 using a wide integer type.

7. Compute actual as the sum of all elements using pointer arithmetic.

8. Display expected - actual as the missing number.

9. Stop.

Test Case Table

TC	Size	Array elements	Expected output	Purpose
1	3	3 0 1	2	Missing value in the middle
2	2	0 1	2	Missing last value
3	9	9 6 4 2 3 5 7 0 1	8	Professor example with nine elements
4	1	0	1	Smallest array; missing upper bound
5	1	1	0	Missing zero
6	5	0 1 2 3 4	5	Missing upper bound
7	5	5 4 3 2 1	0	Reverse order; missing zero
8	4	0 2 3 4	1	Unsorted valid input
9	0	(none)	Invalid size message	Invalid size
10	4	0 1 1 4	Duplicate elements are not allowed	Duplicate input rejected

For Test Case 9, enter only size 0. For Test Case 10, enter size 4 and then the listed elements. The program stops on invalid input.

C Program

```
#include <stdio.h>

void missing_number(const int *nums, int size)
{
    long long expected = (long long)size * (size + 1) / 2;
    long long actual = 0;
    int i;
    for (i = 0; i < size; i++)
        actual += *(nums + i);
    printf("Missing number: %lld\n", expected - actual);
}

int main(void)
{
    int size, i, j;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0 || size > 10000) {
        printf("Invalid size. Enter a value from 1 to 10000.\n");
        return 1;
    }
    int nums[size];
    printf("Enter %d distinct elements from 0 to %d:\n", size, size);
    for (i = 0; i < size; i++) {
        if (scanf("%d", nums + i) != 1 || nums[i] < 0 || nums[i] > size) {
            printf("Invalid element. Values must be from 0 to %d.\n", size);
            return 1;
        }
        for (j = 0; j < i; j++) {
            if (nums[i] == nums[j]) {
                printf("Invalid input. Duplicate elements are not allowed.\n");
                return 1;
            }
        }
    }
    missing_number(nums, size);
    return 0;
}
```

Compilation and Execution

gcc -std=c99 hdoc63.c -o hdoc63

./hdoc63

Compilation and Execution



Test Case 1 - Screenshot

Size: 3 | Elements: 3 0 1
Expected: 2



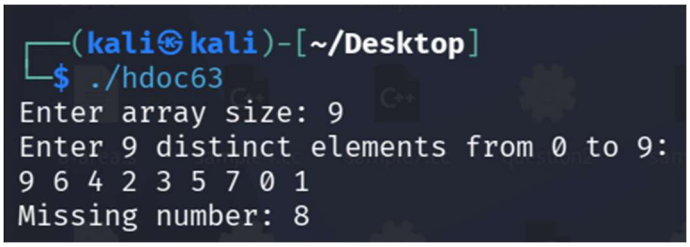
Test Case 2 - Screenshot

Size: 2 | Elements: 0 1
Expected: 2



Test Case 3 - Screenshot

Size: 9 | Elements: 9 6 4 2 3 5 7 0 1
Expected: 8



Test Case 4 - Screenshot

Size: 1 | Elements: 0
Expected: 1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc63
Enter array size: 1
Enter 1 distinct elements from 0 to 1:
0
Missing number: 1
```

Test Case 5 - Screenshot

Size: 1 | Elements: 1

Expected: 0

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc63
Enter array size: 1
Enter 1 distinct elements from 0 to 1:
1
Missing number: 0
```

Test Case 6 - Screenshot

Size: 5 | Elements: 0 1 2 3 4

Expected: 5

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc63
Enter array size: 5
Enter 5 distinct elements from 0 to 5:
0 1 2 3 4
Missing number: 5
```

Test Case 7 - Screenshot

Size: 5 | Elements: 5 4 3 2 1

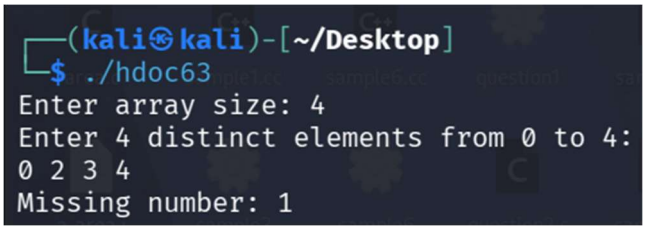
Expected: 0

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc63
Enter array size: 5
Enter 5 distinct elements from 0 to 5:
5 4 3 2 1
Missing number: 0
```

Test Case 8 - Screenshot

Size: 4 | Elements: 0 2 3 4

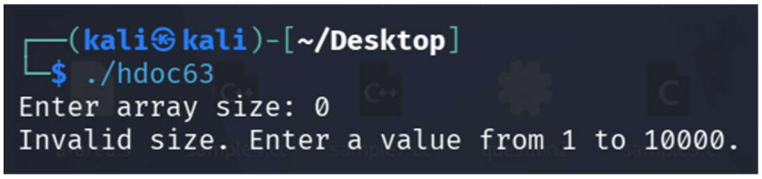
Expected: 1



Test Case 9 - Screenshot

Size: 0 | Elements: (none)

Expected: Invalid size message



Test Case 10 - Screenshot

Size: 4 | Elements: 0 1 1 4

Expected: Duplicate elements are not allowed

